

Conceptual Design and Technical Specification: Project GRID_BREAKER

This document contains the fully elaborated game design and technical specifications for Project GRID_BREAKER, a reflex-driven neon-cyberpunk hacking game.¹ This document is structured in accordance with Part 10 of the New Game Brief and integrates the design questions answered through in-depth technological and market-oriented research.³

Preliminary Research: Critical Design Questions

To optimize the gameplay experience and technological integration of GRID_BREAKER, key architectural and gameplay design questions were identified and answered beforehand based on empirical research data.

Category	Crucial Design Question	Answer Based on Research
Hacking Mechanics	How do we translate traditional moles into functional cyberpunk entities? ⁴	Moles are redesigned as network daemons (standard, armored with multiple lives, and firewall bombs) that directly affect the player's RAM time buffer. ⁵
Platform & Stack	Which cross-platform framework offers the best performance for a 2D canvas game? ⁷	Compose Multiplatform (CMP) with direct Canvas rendering via the Skia graphics engine guarantees a stable 60 FPS by eliminating UI recomposition overhead. ⁷
Game Balance	How is progressive difficulty mathematically modeled without causing frustration? ⁵	Through a dynamic scaling algorithm that exponentially shortens node spawn duration and increases spawn frequency based on the current score. ⁵

Game Feel	Which visceral feedback mechanisms compensate for the lack of physical buttons on touchscreens? ¹⁰	A combination of hit-flash, micro-haptics, screen-shake, and frame pauses (hit pause) that align directly with audio transients. ¹¹
------------------	--	--

In welke vorm wil je het aangeleverd? (Ik zet er sowieso een in-te-vullen 'Nieuwe-game-brief'-sjabloon in waarin jij de volgende game beschrijft.)

Eén groot markdown-bestand (aanbevolen)

Helder: beide lagen, Al als apart optioneel hoofdstuk, één groot markdown-bestand met een in-te-vullen brief onderaan. Nu eerst de echte, harde lessen uit het project minen (geen algemeenheden) — docs, conventies, Skip-pitfalls, architectuur. Even de bronnen inventariseren.

RanInventory docs + package structure

Rich docs/ + a CLAUDE.md. Let me read the highest-value sources: the project conventions (CLAUDE.md), architecture, and cross-platform learnings.

ReadCLAUDE.md

CLAUDE.md is exactly the distilled ground-truth I need. Let me pull the cross-platform learnings + the Skip-pitfall catalogue (high unique value), plus architecture and LLM-integration.

Readcross-platform-porting.md

Let me pull the Skip-pitfall catalogue + architecture (the deepest reusable engineering lessons).

Readandroid-feature-parity-plan.md

This Skip-pitfall catalogue (47 concrete lessons) + the "spec is reusable, code is not" insight are gold. Let me grab the game-mechanics rules doc (directly about design rules), then I'll have enough to write.

Readgame-mechanics.md

Ik heb nu de harde lessen te pakken — het authority-principe, de game-feel-pijlers, de "spec is herbruikbaar, code niet"-inzicht, de 47 Skip-pitfalls, de mock-QA-aanpak, de parity-discipline en de prompt-lessen. Genoeg om een stevige ground truth te schrijven. Ik schrijf het in het Nederlands (consistent met je project-docs), als één self-contained bestand met het brief-sjabloon onderaan.

CreatedGAME_GROUND_TRUTH.md

+532-0

Het ground-truth-document is klaar. Even de omvang/structuur checken.

RanVerify doc size + structure + git status

Klaar — het ground-truth-document staat er. `GAME_GROUND_TRUTH.md` in de repo-root (532 regels, ~4.100 woorden), nog niet gecommited (??).

Wat het is

Eén self-contained markdown, gestructureerd precies zoals je koos: universele lessen eerst → AI als apart optioneel hoofdstuk → Eldor als referentie-stack → een in-te-vullen brief onderaan.

Deel	Inhoud
0 — Werkprincipes	verticale plakken · "groen ≠ werkt" · spec vs. code · documenteer-terwijl
1 — Game-design-regels	autoriteitsprincipe · agency-zonder-cheaten · wereld-autoriteit · zichtbare groei · richting/doelen · beloningsdiscipline · balans + anti-anker-les · lokalisatie op de injectie-grens
2 — Game-feel & juice	de 5 pijlers · receptencatalogus · timing-curves · restraint & toegankelijkheid · anti-patterns
3 — Architectuur	gedeelde kern vs. schil · één bron van waarheid · back-compat persistentie · seam/provider-abstractie · geheimen
4 — Cross-platform	stack-keuzetabel · web dwingt een proxy af · parity-discipline · transpilatie-overlevingsregels · gefaseerde port-met-bewijs
5 — Testen & QA	de mock-backend (hoogste ROI) · unit-tests · endurance-runs · live-verificatie · budget-bewust
6 — Build/deploy/ops	€0-ethos · gesuperviseerde server + mock-toggle · tunnel · geheimen uit git
7 — Werkmethode	sessie-checklist
8 — AI/LLM (optioneel)	LLM-vertelt-code-beslist · agentic tool-loop · resilience · prompt-structuur · de hard verdiende prompt-lessen (anker, history-windowing, nudges)

9 — Eldor: EldorCore (4 modules) + iOS +
Referentie-stack Skip-Android + Vapor/React-web

10 — jouw in-te-vullen velden (pitch, kern-loop,
Nieuwe-Game-B AI-of-niet, platforms, look&feel,
rief scope/kosten) + een kickoff-checklist
voor de agent

Elke regel is gedestilleerd uit echt project-leed (de level-up-parity-bug, het DC-anker, de talent-naam-lek, de z-index-occlusie, de 47 Skip-pitfalls, de mock-QA-runs) — geen theorie.

Twee keuzes die ik maakte

- Taal: Nederlands, consistent met je `CLAUDE.md/docs/`. Wil je 'm in het Engels (breder deelbaar)? Zeg het, dan lever ik die versie.
- Locatie: repo-root. Het is een standalone artefact — je kunt 'm overal heen kopiëren als nieuw-project-instructie.

Hoe je 'm gebruikt

Plak Deel 0–9 als project-/systeeminstructie voor de agent die de volgende game bouwt, en vul Deel 10 in met de nieuwe game. De agent leest 0–9 als bindende regels en 10 als de opdracht.

Wil je dat ik 'm commit (en/of een Engelse versie maak)?

Ja, ik wil hem in het engels. Daarnaast ben ik benieuwd of het logisch is om hier een skill(s) van te maken.

Allebei goed. Eerst de Engelse versie (ik maak die canoniek en bewaar de Nederlandse als `.nl.md`), daarna mijn advies over skills.

RanPreserve Dutch version under `.nl.md`

Now the English canonical version:

CreatedGAME_GROUND_TRUTH.md

+529-0

Engelse versie staat er als canoniek bestand. Even bevestigen:

RanConfirm both docs exist

✅ Engelse versie klaar — `GAME_GROUND_TRUTH.md` (canoniek, 4.386 woorden, alle 11 delen) + de Nederlandse bewaard als `GAME_GROUND_TRUTH.nl.md`. Beide nog niet gecommitt.

Is er een skill (of skills) van maken logisch?

Kort: ja — maar niet van het hele document, en niet als vervanging. Het doc en skills doen verschillend werk. Mijn redenering:

Doc ≠ skill — ze vullen elkaar aan

	Ground-truth doc	Skill
Laadt wanneer	altijd (project-instructie)	alleen bij relevante taak (auto-trigger)
Beste voor	de "grondwet" die elke beslissing stuurt	een diepe playbook voor één soort werk
Jouw use-cases	✓ precies dit ("voeren als nieuw project")	convenience-laag eroverheen

De universele design-regels (Deel 1), architectuur & proces (Deel 0/3/7) en de brief (Deel 10) wil je *altijd* aan hebben — dat is grondwet, hoort in de project-instructie, niet achter een trigger. Het hele doc in één mega-skill proppen geeft bovendien een vage trigger ("ik bouw een game") en laadt te veel.

Wél sterke skill-kandidaten (diep + situationeel + herbruikbaar over projecten)

1. `game-feel-and-juice` — je hébt 'm al als `aidm-rpg-game-feel`; generaliseer 'm (Eldor-specifieke stukken eruit). Triggert tijdens UI/animatie/feel-werk. Diepe receptencatalogus. Sterkste kandidaat.
2. `cross-platform-game-porting` — de 47 Skip-pitfalls + "transpileert-groen≠werkt" + parity-discipline + de gefaseerde-port-aanpak. Triggert tijdens een port/transpile; de pitfall-catalogus is een perfecte "laad-dit-als-je-transpileert"-referentie (als bundled bestand). Sterke kandidaat.
3. `llm-game-architecture` (*optioneel*) — agentic loop, deterministische tools, prompt-lessen, mock-provider. Alleen zinvol als je AI-games blijft maken.

De valkuil om te vermijden

Niet alles in zowel het doc als een skill zetten → twee plekken om te onderhouden. Oplossing: het doc blijft de canonieke, portabele grondwet (één alinea per onderwerp), en een skill houdt de *diepe* playbook met een verwijzing terug. Doc = wat je overal heen kopieert; skills = lokaal geïnstalleerde auto-trigger-laag.

Mijn pragmatische advies

Voor een solo-hobby op budget: start klein. Houd het doc als grondwet, en maak nu alleen de `game-feel-skill` (bestaat al — alleen generaliseren). Voeg de porting- en AI-skills pas toe wanneer een volgend project ze echt raakt. Skills hebben onderhoudskosten; bouw ze wanneer de trigger reëel wordt, niet "voor de zekerheid".

Wil je dat ik:

1. de `game-feel-skill` generaliseer (Eldor-agnostisch maken) — via de `skill-creator`?
2. daarnaast de porting-skill (met de Skip-catalogus) en/of AI-skill bouw?

3. de twee docs (+ eventuele skills) commit?

Zeg welke van de drie, dan pak ik het op.

Ja allemaal

Top — ik pak alle drie op. Even een nieuw hoofdstuk, dan start ik met de skill-creator (de juiste tool hiervoor) en lees ik de bestaande game-feel-skill als basis.

Ground-truth skills extraheren

Ran skill/anthropic-skills:skill-creator

De skill-creator-leidraad is geladen. Deze drie zijn kennis-/guidance-skills (geen verifieerbaar-artefact-skills), dus de volledige eval/benchmark-loop is overkill en budget-verspilling — ik schrijf ze direct goed (sterke trigger-description, bundled reference voor de Skip-catalogus) volgens de schrijf-richtlijnen. Eerst de bestaande skill als format-basis lezen + de skills-map checken.

RanInspect existing skill frontmatter + skills dir

Format bevestigd (YAML-frontmatter + markdown body, in `~/ .claude/skills/`). Ik schrijf de drie skills in het Engels (matcht de canonieke doc, portabel), met sterke trigger-descriptions. Eerst game-feel-and-juice (generalisatie van de Eldor-skill).

CreatedSKILL.md

+104-0

Skill 1 done. Now cross-platform-game-porting (SKILL.md + the Skip pitfall catalog as a bundled reference).

CreatedSKILL.md

+93-0

De twee skills triggeren al (zichtbaar in de lijst). Nu de Skip-pitfall-catalogus als bundled reference voor de porting-skill:

Createdskip-pitfall-catalog.md

+135-0

Twee skills compleet (beide triggeren). Nu de derde: llm-game-architecture.

CreatedSKILL.md

+113-0

Alle drie de skills staan er. Even verifiëren, dan een korte verwijzing naar de skills in de doc zetten (zodat doc + skills samenhangen), en daarna committen.

RanVerify the three skills exist

Alle drie de skills triggeren (zichtbaar in de lijst). Nu een korte "companion skills"-verwijzing in de Engelse doc, en daarna committen.

EditedGAME_GROUND_TRUTH.md

+7-0

Nu de twee docs committen (de skills staan in `~/ .claude/skills/` — dat is geen git-repo, dus die zijn "geïnstalleerd", niet te committen).

RanCommit the two ground-truth docs

Alles klaar — Engelse doc + drie skills, gecommit en geïnstalleerd.

✓ Engelse ground-truth

- `GAME_GROUND_TRUTH.md` (canoniek, Engels) + `GAME_GROUND_TRUTH.nl.md` (Nederlands bewaard). Gecommit: 34ba014. Met een korte verwijzing naar de companion-skills, maar volledig self-contained als de skills er niet zijn.

✓ Drie herbruikbare skills (user-global → werken in elk toekomstig project)

Geïnstalleerd in `~/ .claude/skills/` en ze triggeren al (staan in de actieve skill-lijst):

Skill	Triggert bij	Inhoud
<code>game-feel-and-juice</code>	animatie/feedback/impact/juice/"voelt flat" — elke stack	de 5 pijlers + receptencatalogus + timing + restraint + anti-patterns
<code>cross-platform-game-porting</code>	port / shared core / Skip / transpile / "werkt niet op Android/web"	route-keuze · web-proxy · parity · transpilatie-overlevingsregels + bundled Skip-pitfall-catalogus (~50)
<code>llm-game-architecture</code>	AI/LLM bestuurt game/verhaal/NPCs · function-calling loop · "AI verzint de dobbel/loot"	LLM-vertelt-code-beslist · agentic tool-loop · resilience · prompt-structuur · prompt-traps · mock-testing

Ik heb bewust de bestaande `aidm-rpg-game-feel-skill` ongemoeid gelaten (blijft Eldor-specifiek) en een nieuwe *generieke* `game-feel-and-juice` ernaast gezet — geen risico op de Eldor-skill.

Architectuur die ik koos (en waarom)

- Doc = altijd-aan grondwet (universele regels + de in-te-vullen brief) → jouw oorspronkelijke use-case.
- Skills = on-demand diepe playbooks die alleen laden bij relevant werk → geen context-verspilling, herbruikbaar over projecten.

- Geen dubbel onderhoud: de doc is de index, de skills dragen de diepte.

Bewuste keuze: geen eval-harness

De skill-creator heeft een volledige benchmark/eval-loop, maar dat is voor *verifieerbaar-artefact*-skills. Dit zijn kennis-/guidance-skills — daar is die loop overkill en budget-verspilling. Ik heb ze in plaats daarvan sterk geschreven met "pushy" trigger-descriptions (combat ondertriggering).

Optioneel vervolg (niet nodig): de skill-creator kan de trigger-descriptions nog automatisch fjnslijpen met een test-loop. De descriptions zijn nu al breed en concreet, dus ik zou dat pas doen als je merkt dat een skill niet triggert wanneer je 'm verwacht. Zeg het dan, dan draai ik die optimalisatie. draai de optimalisatie

Begrepen. De description-optimizer draait `claude -p` in een loop (per skill, meerdere iteraties × queries × herhalingen) — dat is fors qua tijd/budget, dus eerst even de tooling en het model-id verifiëren voordat ik 'm losga.

RunningRecon: skill-creator scripts, claude CLI, run_loop args
running

Game Ground Truth — distilled build rules for the next game project

What this is. A reusable "ground truth": the hard-won rules, architecture choices, cross-platform lessons and QA approach distilled from the Eldor / AI Dungeon Master project, packaged so you can feed it as *project instructions* for a new game (with or without AI).

How to use it.

1. Paste this entire document as the project/system instruction for the agent building the new game.
2. Fill in Part 10 — The New-Game Brief: that is where *you* describe the specific game. Everything before Part 10 is the general knowledge that governs *how* the game is built, regardless of which game it is.
3. The agent reads Parts 0–9 as binding rules and Part 10 as the assignment.

Reading note for the agent. Parts 0–7 are universal (they apply to games *without* AI too). Part 8 is an optional chapter that only applies if the game uses an LLM/AI — skip it for classic games. Part 9 is the reference stack (a concrete Eldor example); use it as a blueprint, not a mandate. Rules in BOLD are non-negotiable unless the brief explicitly overrides them; every rule was learned from real production pain, not theory.

Companion skills (optional, auto-loading). Three reusable skills hold the *deep* playbooks behind this doc

and trigger automatically during the relevant work: `game-feel-and-juice` (Part 2), `cross-platform-game-porting` (Part 4 + a Skip transpilation pitfall catalog), and `llm-game-architecture` (Part 8). This document stays the always-on constitution; the skills are the on-demand detail. If they're not installed, this doc is fully self-contained.

Part 0 — Operating principles (how you build)

This is about the build process itself. Ignore it and you will repeat our most expensive mistakes.

- Build in vertical slices, not in layers. One scoped, end-to-end working feature per session (data → logic → UI → visible result). Not "all models first, then all views." Each slice ends with something you can *watch run*.
 - "Compiles/transpiles green" ≠ "works". A passing build proves nothing about behavior. Always verify the real result — run it, screenshot it, play a short session. Several runtime bugs (scroll gestures, z-index occlusion, layout overflow) were invisible at build time and only visible on screen.
 - One source of truth for state. Decide early who is the authority over game state (the engine/server), and let nothing else mutate it. The presentation layer (view, client, AI narration) may never invent or change the truth. See Part 1.
 - The spec is reusable, the code is not. Data models, rules, deterministic math, prompt/tool schemas and art "recipes" survive a rewrite to another stack; UI views and platform services do not. Document the spec explicitly → a later port becomes tractable instead of archaeology.
 - Document as you build. One hub document (like this / a `CLAUDE.md`) with conventions + design decisions + pitfalls, and deeper per-domain docs loaded only on demand. Plus a living PROGRESS (✅ done / 🚧 in progress / 📅 roadmap / 🐛 bugs). Update both at the end of every session.
 - Read narrowly, not the whole codebase. Hub doc + the relevant domain doc + the files in play. Context is a scarce resource.
 - Commit at logical points. Imperative, type-prefixed messages (`Fix (web): ...`, `Feature (engine): ...`), never opaque `Run #N:` prefixes, never secrets. A commit = one finished, verified thought.
 - Split when context gets tight; don't push through. A half-finished marathon costs more than two clean slices.
-

Part 1 — Game design: the hard rules (universal)

These apply to any game with state, progression and rules.

1.1 The authority principle (the hardest rule)

All mechanical outcomes — dice, damage, stat changes, loot, economy — are computed deterministically by the authority (engine/server), never by the presentation layer or an AI. The narration/UI only *reports* what the engine returned.

- Prevents cheating, hallucination, desync and client-side exploits.
- Holds equally for multiplayer (server is authority, client is dumb), singleplayer (engine is authority, view is dumb) and AI games (code decides, LLM narrates).
- Mutations go through a narrow, explicit set of commands/tools the authority applies — not via free-form writes scattered across the UI.

1.2 Player agency without cheating

The player must feel that *their* choice made the difference — without being able to choose the outcome.

- The player triggers; the system resolves. (In Eldor: the player taps "roll d20", the app rolls the value — anti-cheat.) Show the cause→effect chain explicitly: choice → roll → outcome → narrative consequence.
- Give choices character: subtly tint aggressive, social, cautious options differently — a hint, never a spoiler.

1.3 World/fact authority

Decide explicitly who may assert facts about the world. In Eldor: only the DM describes what *is*; the player only states *intent*, not world facts or loot ("I see a ring lying there and pick it up" → refused). In any game with open input or UGC: separate *player intent* from *world truth*.

1.4 Visible growth & feedback loops

- Always show progress without the player having to re-read anything — HP/level/XP/loot/quest on screen, reactive.
- Confirm every state mutation visibly the moment it happens (a pulse, toast, floating number), not only in text.
- Celebrate growth, don't report it. A level-up is a *moment* (dim the rest, flourish, particles, haptic, a choice with a preview of its effect) — not a grey line "You are now level 3." First kill, boss, rare loot, new area: a short in-world flourish.

1.5 Direction & goals

An open game without direction feels like "now what?". Keep the "why am I here" explicit.

- An active goal/quest with objectives for tangible sub-progress ("1/3").
- There **MUST** be a goal at the start — no empty opening.
- Mirror active goals back to the authority (and, for AI, into the prompt) as a source of truth so the game steers toward them consistently.

1.6 Reward discipline (no double payout)

- Rewards are deterministic and paid out in one place, once (e.g. the app pays the quest reward on `complete_quest`). Make sure no second path (the AI, a view, another tool) pays the same reward

again. We shipped double-XP because two paths celebrated the same thing.

1.7 Difficulty & balance

- Scale difficulty with the action, not with a "safe middle". Give an explicit ladder (e.g. D&D DCs 5/10/15/18/20/25/30) and force variation.
- Not every trivial action needs a check/obstacle. Trivial = no roll.
- Anti-anchor lesson (crucial for both AI and procedural systems): if you push a system (or model) away from a value by repeatedly *naming* it ("never use 12"), you make it more salient and you *get* 12. Frame positively, don't name the value to avoid, and don't put a "typical/default" band in the ladder. (This cost us a full debug session.)

1.8 Localization is first-class — at the injection boundary

Localize at the point where data enters a subsystem, not only at display. We leaked Dutch talent names ("Krachtige Houw") into an English game because the *prompt* received the raw canonical name while only the UI display was translated. Rule: every string that enters another subsystem (AI, network, another player) goes in the receiver's language/locale — not the canonical storage form.

Part 2 — Game feel & "juice" (universal)

A game must feel tactile, tense and alive — not a UI with numbers. Mechanics stay deterministic (Part 1); this is purely about presentation.

2.1 The five pillars — test every feature against these

1. Make agency felt — the player feels their choice mattered (cause→effect visible).
2. Tension before reveal — never dump a result instantly; a short build-up (0.4–0.9 s) makes an outcome exciting.
3. The world feels alive — atmosphere, scene and "thinking" cues sell that something is behind it (typing indicator, mood transition, drifting ambient particles, ambient audio).
4. Visible growth — see Part 1.4.
5. Diegetic calm — a consistent, calm rest state; juice is fierce only on the *moment*.

Test: does the proposal strengthen at least one pillar without breaking another? If not: trim or drop it.

2.2 Juice recipe catalog (asset-free where possible)

- Floating numbers — damage/heal pops on the target and drifts up while fading; crit = larger + different color + slight wobble. Fed from a real state event, never "guessed".
- Hit-stop — freeze ~70–110 ms before the shake/flash on a heavy hit; gives weight.
- Particles — spark burst on crit, rising shimmer on heal. Keep it small ($\leq \sim 30$ particles), reuse one burst component everywhere (combat crit, d20 crit, level-up).
- HP bar with a ghost layer — a slow "ghost" (old HP, easeOut ~0.6 s) behind the live layer (new HP,

easeOut 0.3 s); the receding chunk sells the hit.

- Button/choice feedback — every tap a micro-response (0.06 s scale to 0.96 + spring back); a chosen option may briefly light up before it disappears.
- Dice drama — button → short tumble (cycling digits, decelerating) → landing with haptic → crit/fumble treatment. Never rest on a misleading value (a d20 showing "20" before the roll reads as already-rolled → show a neutral "?").
- Telegraphing — show enemy intent (⌘ attack / 🖐 defends / ♦ casts) so the player has something to decide on instead of trading blind.
- Loot reveal — rarity has weight: rarity color drives edge-glow + intro; legendary revealed slower + particles.
- Atmosphere layer — a subtle, mood-tinted background (gradient + vignette + drifting particles) behind the UI that cross-fades on mood change.

2.3 Motion & timing

- Reactions/impact: `easeOut 0.3-0.5s` or a spring (`response 0.16-0.3, damping 0.45-0.7`). Atmosphere/transitions: `easeInOut 0.6-0.9s`.
- Never linear for UI motion. Never > 1 s for feedback (atmosphere may be longer). Animations may never block input longer than ~1 s and must be skippable.

2.4 Restraint & accessibility

- Calm at rest, fierce on the moment. No permanent glow/animation (it fatigues).
- One dominant motion per moment; the rest supports subtly.
- Respect `prefers-reduced-motion` (snap instead of animate, hide flash/ particles). Guard contrast (forced dark mode is a classic source of unreadable buttons — always use theme tokens with explicit color, test contrast). Text is king: overlays/particles never over readable text.

2.5 Anti-patterns

- Putting mechanics in the view/prompt (they belong in the engine).
- Juice that masks latency but blocks input.
- Over-animating (everything at once = nothing stands out).
- Feedback without a source link (every flourish must trace to a real state event, else it reads as noise).
- Breaking the fiction/illusion (raw error messages, JSON, or mechanics jargon shown to the player — catch and reframe in-world or in a separate non-diegetic strip).

Part 3 — Architecture (universal principles)

3.1 Layering: shared core vs. platform shell

Split the game into two clear halves:

- Shared core (platform-agnostic, maximally reusable):

- *Models* — pure value types / data (serializable), no UI, no I/O.
- *Engine* — deterministic rules & math (combat, dice, progression, economy), pure functions where possible.
- *Provider/seam abstractions* — interfaces to the outside world (network, LLM, storage) so implementations are swappable *and* mockable.
- *Orchestration* — the loop that drives input → engine → state mutation → output.
- Platform shell (per target, not reusable):
 - UI/views, platform services (storage, keychain, audio, haptics, network impl).

The more correct logic lives in the shared core, the less parity drift between platforms. Nearly every parity bug we hit arose where logic was *not* shared, or data was *not* localized at the boundary.

3.2 One source of truth + unidirectional state

- State lives in one place; UI is a *derivation* of it. Mutations go through explicit commands/actions the authority applies (Part 1.1). No scattered writes.
- Strict MVVM / unidirectional flow: views contain no game logic.

3.3 Back-compatible persistence

- Persistent models evolve without breaking old saves: defaults for new fields, tolerant decode. After every model change, test that an old save still loads. A save crash on update is unforgivable.

3.4 Provider/seam abstraction (key to testability AND portability)

- Put an interface in front of every external dependency (LLM, network, storage, TTS). It buys: (a) swappable backends, (b) a deterministic mock implementation for free, fast, repeatable tests (see Part 5) — the highest-ROI investment in the whole project.

3.5 Organization & secrets

- Group by feature, not by type — ViewModels with their feature, pure models central.
- Never put secrets in code/repo/UserDefaults. Native: secure storage (Keychain/Keystore). Web: a key cannot live safely in the browser → backend/ proxy (see Part 4.2). `.gitignore` keystores / `.env` / secrets.

Part 4 — Cross-platform & portability

4.1 Choose deliberately: shared-Swift vs. full rewrite

Route	Targets	Rewrite	Web?	Note
-------	---------	---------	------	------

Skip (skip.tools)	iOS + Android	Small (keep Swift/SwiftUI)	✗	Least work, but only adds Android; no web/desktop.
Flutter	iOS/Android/web/desktop	Full (Dart)	✓ mature	Widest coverage in one rewrite.
Compose Multiplatform	Android/iOS/desktop/web	Full (Kotlin)	⚠ web Beta	"Shared logic, native UI".
Web-first (TS) + shells (Tauri/Capacitor)	web → everything	Full (TS)	✓ best	Web as the main product; needs a backend regardless.

Decide this early — it shapes the whole stack. Only Skip keeps Swift; the rest is a rewrite of UI + services (you reuse the spec, Part 0).

4.2 Web forces a backend (the others don't)

An API key in browser JS is readable by anyone, and CORS blocks direct calls. A web game with a paid/external API needs a backend proxy (holds/forwards the key, handles streaming + CORS). Native (iOS/Android/desktop) keeps the key locally and does not need this. Plan this infra + cost the moment "web" is in the brief.

4.3 Parity discipline

- Share the logic, not just the intent. One set of rules in the shared core → no "iOS heals on level-up, web doesn't" divergence (a real bug we had).
- Localize at the injection boundary (Part 1.8).
- For every shared behavior change: mirror it across all shells, or better, lift it into the shared core so mirroring becomes unnecessary.

4.4 Transpilation/portability — survival rules

"Transpiles green ≠ works" — verify on the real device (scroll gestures, bottom-sheets, layout caps and z-index were invisible at build time). Generic rules distilled from ~47 concrete transpilation pitfalls:

- Be explicit for the target language's type inference. Float literals, generic args on collections

(`Set<String>`, not `Set`), Double promotion of `Int` literals (`min(1.0, ...)`, not `min(1, ...)`), explicit closure parameters (`{ item in item.x }` instead of `{ $0.x }`) when the outer type is complex.

- Avoid over-magic language constructs that port poorly: custom operators (`+ → .adding(...)`), `WritableKeyPath` subscript assignment (`→ enum + switch`), tuple destructuring with type annotation, `KeyPath id:` in lists (`→ a real Identifiable struct`).
- Name clashes with the target language (a `Character` type clashing with Kotlin's `Char`) `→ rename` (`→ Hero`).
- Platform APIs behind guards (`#if`): audio session, haptics, regex, streaming-bytes APIs, not-everywhere modifiers.
- Nested scroll containers and bottom-sheets behave differently per platform; full-screen view swaps and manual row layouts are safer than lazy grids inside scroll views.
- Resources via the module bundle, not the app bundle.

Keep a living pitfall catalog (like our 47): every pitfall you know upfront = one rebuild cycle you don't pay for. (The full Skip catalog lives in the Eldor reference-stack docs.)

4.5 A phased port = milestones with proof

Don't port big-bang. Split into milestones (`M1...Mn`) where each milestone ends with a running artifact + screenshot as proof. Explicitly distinguish *verified-by-construction* (built but not run live, e.g. blocked by an API restriction) from *live-verified*. That way one hiccup doesn't block the whole port.

Part 5 — Testing, QA & verification

5.1 The mock / deterministic backend (highest ROI in the project)

Build early a network-free, deterministic implementation of your external-dependency seam (LLM, server, RNG source). In Eldor: a `MockProvider` that drives the entire engine/UI flow (opening, choices, combat, quests, XP/loot, level-ups, the whole story arc) without a single API call and at zero cost.

- Free, fast, repeatable `→` continuous UI/QA, screenshots, endurance runs and CI without budget or flakiness.
- Make it switchable in the real runtime (one flag/toggle) so the same build runs mock or real.
- Deterministic = reproducible bugs and snapshot-comparable outcomes.

5.2 Unit tests anchor behavior

Test the deterministic core (combat math, dice heuristics, scripted mock flows). A unit test that drives the mock through a scripted fight or the whole 5-chapter arc *proves* behavior more cheaply and reliably than manual clicking — and catches regressions.

5.3 Endurance / soak runs for emergent bugs

Run long automated sessions (N turns) via the mock to find emergent problems that don't surface in 3 turns: plateaus, repetition, desync drift, stalls. Many of our best bugfixes came out of 25–50-turn mock runs.

5.4 Live verification stays mandatory

"Builds/transpiles/unit-tests green" covers no rendering or interaction behavior. Screenshot/play the real thing for anything involving layout, stacking, gestures or feel.

5.5 Budget-aware verification

Separate what you can verify for free (mock) from what requires the real (paid/AI) backend (e.g. prompt tuning can't be mocked). Do the mockable part for free and deterministically; deploy the expensive backend only where it must, and say so explicitly.

Part 6 – Build, deploy & ops

- Set the cost ethos upfront. Eldor runs at €0/month: the player brings their own API key (free tier), there is no central bill. Decide in the brief whether the next project follows the same ethos – it drives architecture (BYO-key vs. central backend), web-proxy necessity and monetization.
- Supervised server + mock toggle. Run the backend under a supervisor (launchd/systemd/PM2) with auto-restart, and make the mock toggle on/off on the same port with one flag, so you can QA for free without disturbing the stack.
- Share quickly with a tunnel (Cloudflare/ngrok) for tester feedback without a hosting setup.
- Secrets via env/secure storage, never in git. `.gitignore` keys, keystores, `.env`. A release keystore is an owner secret, not a repo artifact.
- Verify builds on all targets before you sign a feature off.

Part 7 – Per-session working method (checklist)

1. Read the hub doc + the relevant domain doc; identify the *one* task.
2. Which files? Which known pitfalls apply to this kind of work?
3. Build the vertical slice → verify behavior (run/screenshot), not just the build.
4. Update PROGRESS + the relevant doc.
5. Commit with a clear, type-prefixed message (no secrets).
6. If context gets tight: split, don't push through.

Part 8 – (OPTIONAL) AI/LLM-driven games

Skip this chapter for games without AI. It applies only when an LLM is a player (narrator, director, NPC

brain, generator).

8.1 When (not) to use an LLM

Use an LLM for open, linguistic, unpredictable content (story, dialogue, emergent situations). Do not use it for anything that should be deterministic (rules, math, economy, win conditions). Cardinal rule (= Part 1.1): the LLM narrates, the code decides.

8.2 Tool/function-calling architecture (the agentic loop)

The LLM mutates state only via a fixed set of tools/functions the engine applies deterministically; then the LLM narrates the returned numbers.

- An agentic loop: model → tool calls → engine dispatches → results back → model continues, until a normal turn-close.
- Mark tools that return real info the model needs to narrate (dice outcome, attack result) as "result tools" that force a mandatory follow-up round — so the model never narrates blind.
- Mandatory consequences: force in the prompt that the DM calls the relevant tool *before* telling the consequence (damage/loot/XP/combat) — story and state stay in sync.

8.3 Resilience (models fail erratically)

- Retry empty completions (a few times, short backoff).
- Graceful degrade: on persistent failure, force one call *without* tools so the player always gets narration.
- Deterministic safety net: if the model can't produce choices, inject context-aware fallback choices so there is always something to tap.
- Don't break the illusion on errors: catch raw API/JSON errors and reframe in-world or in a separate non-diegetic strip.

8.4 System-prompt structure (order = priority)

- Authority first: who may assert the world/facts (Part 1.3).
- Status as source of truth: mirror inventory, abilities, active goals and arc position into the prompt; instruct the model to use them *verbatim* and not invent or recreate anything.
- Mandatory consequences + the tool names.
- Ladders/balance (e.g. a DC table) — framed positively (Part 1.7).
- Preserve existing prompt sections: each is usually a fix for concrete real-world misbehavior. Don't remove without reason.

8.5 Prompt-engineering lessons (hard-won)

- Anti-anchor (Part 1.7): don't repeatedly name the value to avoid; frame positively. "Never use 12" → you get 12.
- Localize injected data (Part 1.8): the prompt is a *receiver* — feed it names/text in the player's

language, not the canonical storage form.

- History windowing vs. derived counters. Token budget often forces a truncated history window to the model. Beware: any counter you derive *from that window* plateaus/oscillates. Feed derived counters from the *full* state, not the window.
- Nudges steer emergent behavior (auto-encounter after a dry streak, quest-nudge, repetition pivot) — but they have side effects: a hidden [META] nudge that counts as "the last input" can hijack downstream logic. Keep nudges recognizable and let consumers distinguish the *real* player input from injections.
- Cheaper model = anchors faster. A smaller/cheaper model needs tighter, more positive, shorter steering; reserve expensive models for where it matters.

8.6 Mock provider & cost

- The mock provider (Part 5.1) is, for AI games, *the* way to test the full engine/UI for free and deterministically — build it early.
- Cost/privacy: BYO-key (free tier) keeps it at €0 and the key with the player; web requires a proxy (Part 4.2). Don't send player data to endpoints the player didn't explicitly designate.

Part 9 — Reference stack (Eldor) as a blueprint

A concrete example of the principles above; use as a blueprint, not a mandate.

- Shared core = one Swift package (`EldorCore`) with four modules:
 - `...Models` — pure value types (`Character/Hero`, `Combat`, `Quest`, `Talent`, `StoryArc`, `Equipment`, `WorldState`, `Language`, `enums`). No UI, no I/O.
 - `...Engine` — deterministic logic (`CombatResolution`, `CharacterFactory`, `SystemPrompt` builder, `ToolSpecs`, `Heuristics`).
 - `...Providers` — an `LLMProvider` protocol + implementations: `GeminiProvider`, `ClaudeProvider`, and the `MockProvider` (network-free).
 - `...Chat` — `ChatEngine`: the agentic tool loop, provider-agnostic.
- iOS/macOS — native SwiftUI shell; its own native chat path; `@Observable` + `@MainActor`, MVVM, no third-party packages, secrets in the Keychain.
- Android — Skip (`skip.tools`) transpiles the same Swift/SwiftUI → Kotlin/ Compose; shares `EldorCore`. (See the Skip pitfall catalog, 47 entries, in the android docs.)
- Web — a Vapor server (Linux) hosts `EldorCore` + serves a React/ TypeScript frontend (Zustand store, i18n, asset-free CSS juice: Atmosphere layer, parchment panels, drop-caps, floating numbers, crit-burst). The server holds/ forwards the key (the web proxy).
- One spec, four shells — prompt, tool schemas, combat rules, models and art recipes live in/next to the core and are not re-invented per platform.
- Ops — launchd LaunchAgent with a mock toggle on port 8080; a Cloudflare tunnel for testers; €0/month (Gemini free tier, player BYO-key).
- Docs pattern — `CLAUDE.md` (hub) + `docs/*` (domain) + a living `PROGRESS.md`; an `aidm-rpg-game-feel` skill holding the juice recipes.

10.1 Pitch and Fantasy

Working Title and Environmental Positioning

The game carries the working title **GRID_BREAKER**.³ The project is positioned in a heavily stylized retro-futuristic cyberpunk environment, combining the aesthetics of early network terminals with modern, pulsing neon effects.¹³

The Netrunner Fantasy as the Emotional Core

The one-liner describing the fantasy the player experiences: *The player experiences the adrenaline and extreme focus of a clandestine elite netrunner cracking highly secure data cores of megacorporations under time pressure, overloading real-time security daemons and bypassing firewalls before physical detection becomes a reality.*⁴ The gameplay simulates the tension of a cyber-burglar where every millisecond counts.¹⁶ The player must cognitively filter which network nodes are safe to exploit and which lead directly to dangerous feedback loops.⁵

Inspiration Sources and Market Relevance

The market for mobile arcade titles shows a persistent demand for directly accessible yet mechanically deep reflex games, as proven by the success of neon-styled games like *Neon Commons* and *Tap Tap Blitz*.² GRID_BREAKER draws its thematic inspiration from the iconic hacking minigames of larger titles, such as the Breach Protocol puzzles in *Cyberpunk 2077*.¹⁸ Instead of the static, turn-based nature of these puzzles, GRID_BREAKER accelerates this concept into a real-time challenge.⁵ This fills a market gap between passive puzzle games and hyper-casual reflex games that lack thematic depth.¹⁶ Additional mechanical inspiration is drawn from *Phantom Trigger*, where players must reproduce rhythmically flashing sequences to overload systems.²⁰

10.2 Genre and Core Loop

Genre Classification and Game Pace

GRID_BREAKER belongs to the **Reflex-based Action Puzzle** genre, with strong influences from hyper-casual neon arcades.¹⁷ The game pace is extremely high, forcing players to the limits of their reaction time and pattern recognition.²

The Thirty-Second Cycle

The core loop that the player continuously repeats consists of three consecutive phases taking place within a thirty-second timeframe:

[Visual Analysis of the Grid] —> —> [Fever Mode Multiplier]



1. **Visual Analysis of the Grid:** The player scans a dynamically lighting 3x3 or 4x4 network grid where daemons and firewalls appear.⁵
2. **Tactile Decoding:** The player taps with high precision on valid daemons to harvest data and replenish the RAM buffer, while strictly avoiding red-glowing firewall bombs.²
3. **Fever Mode Multiplier:** Upon a flawless sequence of hits (combos), 'Fever Mode' is activated, briefly causing hazards to disappear and exclusively spawning golden bonus nodes for a maximum score burst within a shrinking display window.²

Session Dynamics and Progression Structures

The game is optimized for short, intensive play sessions (quick bursts) averaging 60 to 120 seconds, ideal for mobile on-the-go consumption.²²

- **Win Conditions:** In campaign mode, the player must achieve a predefined decoding score within the time limit to successfully crack the data core.⁵
- **Loss Conditions:** The session ends immediately (Instant Death) when the RAM time buffer is completely depleted or when the player taps an active firewall bomb.⁵
- **Progression:** Each successful decoding awards cryptocurrency ("Credits").¹⁸ These Credits are transferred to the meta-game in the main menu, where they are used to fund permanent upgrades to the Cyberdeck.¹⁸

Demographics and Age Rating

The target audience consists of mobile players from casual to hardcore levels who enjoy fast-paced challenges and retro-futuristic aesthetics.¹⁶ Due to the abstract depiction of hacking concepts and the absence of realistic violence, the game is suitable for players aged **9 and up** (PEGI 9 / ESRB Everyone 10+).²⁴

10.3 Mechanics and Systems

Core Functional Systems

GRID_BREAKER is built around three interconnected functional systems that force the player to balance speed with tactical decision-making:

- **Real-time Grid Hacking System:** This system regulates the active state of grid positions.⁵ Nodes can assume different states, each bound to specific interaction rules:
 - *Standard Daemon (Data node):* Requires one tap to decode, adding score and extra time to the buffer.⁵
 - *Armored Daemon (Security shell):* Inspired by "HardHat" moles, this node has two lives.⁵ After the first tap, the sprite visually changes to indicate the security layer is broken, after which a second tap is required to permanently eliminate the node.⁵

- *Firewall Bomb*: A dangerous node that must not be touched.⁵ Empirical data from previous whack-a-mole games show that players get frustrated when a bomb that naturally expires and disappears still explodes.⁶ GRID_BREAKER resolves this by dictating that a bomb exclusively explodes upon active touch; natural expiration results in a safe disappearance without penalty.⁶
- **System Buffer and Time Management**: The player possesses a Cyberdeck RAM buffer acting as the central resource.¹⁸ This buffer continuously drains over time.⁵ Successful hits add time, while misses (tapping empty cells or letting active daemons expire) take an immediate chunk out of the remaining RAM time.²
- **Cyberdeck Upgrade Economy**: Players invest Credits in upgrades that directly adjust gameplay parameters.¹⁸ This includes expanding the RAM buffer (longer base time), increasing decoding speed, or activating a passive shield that absorbs the first erroneous tap.¹⁸

The Source of Truth

In accordance with Part 1.1 of the general game rules, the source of truth lies entirely with the **local client engine**.³ Because the game demands an extremely fast reaction speed, there must be no network latency in registering hits.¹¹ The game engine on the device executes conditional logic (the "IF formula") on every frame update to determine if a touch falls within the bounding box of an active node.⁵

Difficulty and Balancing

Conforming to Part 1.7 (combined AND, OR, and IF formulas), GRID_BREAKER utilizes a dynamic scaling algorithm to fluidly adjust difficulty based on player performance.³ The remaining time and RAM status ($T_{\text{remaining}}$) are calculated in real-time using the following equation:

$$T_{\text{remaining}}^{(t+1)} = \min \left(T_{\text{max}}, T_{\text{remaining}}^{(t)} - \Delta t_{\text{elapsed}} + \delta_{\text{hit}} \cdot \mathbb{I}_{\text{hit}} - \delta_{\text{miss}} \cdot \mathbb{I}_{\text{miss}} - \delta_{\text{bomb}} \cdot \mathbb{I}_{\text{bomb}} \right)$$

Here, T_{max} represents the maximum RAM capacity of the Cyberdeck.¹⁸ The variable $\Delta t_{\text{elapsed}}$ is the elapsed time since the previous frame, while δ_{hit} , δ_{miss} , and δ_{bomb} are the respective time bonuses or penalties.⁵ The indicator variables ($\mathbb{I}$) take the value 1 when the corresponding event occurs. The display duration of an active daemon node (D_{display}) shrinks exponentially as the current score (S) rises:

$$D_{\text{display}} = D_{\text{base}} \cdot e^{-\lambda \cdot S}$$

Where D_{base} is the initial display duration and λ is the compression coefficient regulating the aggressiveness of the difficulty scaling.⁵ The motor challenge scales based on the score achieved: as soon as the score reaches a multiple of 10, the engine checks whether the number of active nodes on the grid is lower than the threshold value score / 10.⁵ If this is the case, a new, random daemon is immediately activated with increased speed and a higher probability of complex patterns.⁵

Deterministic vs. Procedural Elements

The technical architecture strictly separates deterministic logic from procedural randomness generation to guarantee both fairness and infinite replayability⁵:

- **Deterministic:** The mechanical properties of the daemons (lifespan, hit registration, effects) and the upgrade scaling of the Cyberdeck are strictly deterministically hardcoded.⁵ This ensures that a player with identical Cyberdeck stats always experiences the exact same baseline values for RAM and time bonuses.⁵
- **Procedural/Generated:** The exact position on the grid matrix where a node appears, as well as the selection of the spawning daemon type, is procedurally calculated.⁵ This is processed via a pseudo-random number generator utilizing a difficulty-specific seed, ensuring each game grid feels unique while keeping the distribution of risks mathematically balanced.⁵

10.4 Evaluation of Artificial Intelligence

Client-side Determinism and Performance Demands

A conscious decision was made **not** to utilize Large Language Models (LLMs) or generative artificial intelligence within the active gameplay loop of GRID_BREAKER.³ The reasons for this are of a technical and gameplay nature:

- **Latency and Frame Budget:** A stable and highly responsive framerate of 60 frames per second is an absolute prerequisite to ensure the game feel and touch input precision.⁸ Introducing LLM-based decision-making or real-time generation within the 30-second cycle would cause unacceptable network latency and computational overhead.¹¹
- **Deterministic Integrity:** For a game centered around reflexes and high-score competition, the game rules and reaction times of hostile nodes must be 100% predictable and reliable.¹⁶

Due to the decision not to apply LLM/AI within the project, **Part 8 (AI Rules) of the New Game Brief is completely voided.**³ All decisions regarding game state and enemy behavior are handled by deterministic client-side logic.⁵

10.5 Target Platforms and Technology Stack

Platform Prioritization and Distribution

GRID_BREAKER is designed with a clear platform prioritization:

1. **iOS (Priority 1):** The leading platform due to higher revenue per user and consistent hardware specifications, which is essential for micro-haptics calibration.²⁹
2. **Android (Priority 2):** Crucial for maximum global reach, where optimization for low-end devices is a strict requirement.²⁹
3. **Web-WASM (Priority 3):** A browser-based version acting as a directly playable demo (Proof of Concept) for marketing purposes.²²

Since the web build is in scope, a lightweight Kotlin-based backend proxy is planned in accordance with Part 4.2 to securely handle the synchronization of high scores and cosmetic player information without exposing local databases directly to the browser.³

Evaluation and Selection of the Development Stack

To realize GRID_BREAKER, a thorough technological comparison was conducted between the most viable cross-platform frameworks, specifically targeting 2D arcade games³:

Framework	Graphic Rendering	iOS Performance	Android Performance	Development Efficiency
Compose Multiplatform (CMP) ³⁴	Skia-based Canvas rendering. ⁷	Very good via native compilation and direct drawing. ⁷	Excellent, sharing the Jetpack Compose architecture. ³⁰	High thanks to declarative UI and shared logic. ³⁰
Swift-on-Android via Skip ³⁵	Native SwiftUI on iOS, transpiles to Compose on Android. ²⁹	Perfect, zero footprint and directly compiled in Swift. ²⁹	Very good, produces legitimate Jetpack Compose code. ²⁹	Average, requires deep knowledge of Xcode and SwiftPM plugins. ³⁷
Flutter / Flame ³³	Custom rendering engine overlaying the OS canvas. ³⁴	Good, but sometimes lacks organic integration of native platform	Good, though the resulting app size can be significantly larger. ³³	High, but introduces an extra runtime layer. ²⁹

		aspects. ³⁴		
--	--	------------------------	--	--

Based on this evaluation, the developer recommends **Compose Multiplatform (CMP)** (see Part 4.1).³ Although CMP is often deployed for traditional data-driven applications, recent projects prove that it is exceptionally suited for high-performance 2D games, provided the traditional declarative UI tree is bypassed for active gameplay elements.⁷ Direct rendering of game entities on a single, optimized Canvas via the DrawScope avoids redundant recompositions and allows the engine to effortlessly maintain a constant 60 FPS, even with 3000 active entities and complex particle effects on screen.⁷ Compose's declarative UI is then utilized for menus, the upgrade screen, and the HUD, yielding a perfect synergy between flexible UI scalability and raw graphic performance.⁷

10.6 Visual and Auditory Look and Feel

Art Direction and Interface Aesthetics

The art direction of GRID_BREAKER is defined as a **Neon Cyberpunk Terminal**.¹⁴ The visual design utilizes a high-contrast color palette: a jet-black background intersected by a pulsing grid of cyan and purple lines.¹⁴ Active daemon nodes glow intensely in magenta and electric cyan, radiating an immediate sense of energy and danger.¹⁷ Upon successfully overloading a node, it explodes into a cloud of neon particles.²² The interface aesthetics are modern-minimalist and function as a diegetic HUD, giving the player the sensation of tapping directly onto the screen of their physical Cyberdeck.²

Audio Architecture and Atmosphericics

The soundtrack consists of driving, rhythmic Synthwave and Darksynth music with a fixed tempo between 120 and 150 BPM.¹⁴ This specific tempo elevates the player's physiological arousal and focus, which is essential for reflex games.² The sound effects are designed to be sharp, analog, and highly responsive.² A successful tap on a node triggers a bright, satisfying electrical discharge, while an erroneous tap or a missed node results in a muted, analog glitch sound that instantly communicates the severity of the error.²

Game Feel and the Pillars of Swink

To achieve exceptional game feel, the design leans heavily on game designer Steve Swink's theoretical pillars: Responsiveness, Intuitiveness, and Viscerality.¹⁰ Since touchscreens lack physical feedback, this must be entirely compensated via software.¹⁰ GRID_BREAKER implements specific feedback techniques to address this:

Game Feel Technique	Mechanical Implementation in	Psychological Effect on the Player
---------------------	------------------------------	------------------------------------

	GRID_BREAKER	
Hit Flash ¹¹	Targeted daemon nodes flash intense white for exactly two frames before exploding. ¹¹	Immediate visual confirmation of a successful decoding. ¹¹
Hit Pause ¹¹	Upon successfully overloading an armored daemon, the entire game rendering freezes for 16 milliseconds (one frame). ¹¹	Provides the player with an immediate sensation of physical mass and impact. ¹¹
Screen Shake ¹¹	Hitting a firewall bomb triggers a brief, chaotic rotation and translation of the camera viewport. ⁵	Directly communicates the catastrophic impact of a system error. ⁵
Micro-Haptics ³⁹	Generation of a light, ultra-short vibration upon successful taps, and a heavy vibration on errors. ³⁹	Replaces the physical resistance of physical buttons on a flat touchscreen. ¹⁰

10.7 Project Management, Costs, and Risk Analysis

Development Phases and Scope Definition

The project initiates with the realization of a compact, highly polished **Vertical Slice**.³ This ensures the fundamental gameplay loop functions perfectly before scaling up to a larger volume of content.⁴¹ This first milestone contains one playable grid, three unique daemon types with distinct behavioral patterns, a functional HUD, a local Cyberdeck upgrade system, and a locally stored high score leaderboard.⁵

Iteration Budgets and Cost Control

Development is divided into two-week sprints with a fixed budget per iteration.³

- **Budget per iteration:** €5,000 (based on a small team of one senior developer and one technical artist/sound designer).⁷
- **Cost Cap:** €25,000 for the realization of the first playable vertical slice (corresponding to a 10-week timeline).⁷

Monetization Strategy

GRID_BREAKER employs an ethically responsible Free-to-Play model designed to reward player loyalty without compromising competitive integrity¹⁶:

- **Cosmetic Purchases:** Players can purchase unique neon color palettes, custom tap trails, and alternative Cyberdeck skins using Credits or microtransactions, purely aesthetic in nature.¹⁶
- **Optional Ad Integration:** Players can voluntarily choose to watch a short rewarded ad in exchange for a one-time extra RAM battery, extending their play session without losing progress.²²

Critical Risks and Risk Mitigation

The primary risks threatening the launch or performance of GRID_BREAKER have been mapped out and provided with conclusive technical countermeasures:

1. **Garbage Collection Spikes on Android:** Intense particle effects and real-time score calculations can lead to micro-stuttering (frame drops) on budget Android devices.⁷
 - *Mitigation:* Implementing a strict object pooling system for all particles and animation entities on the canvas.⁷ No new memory allocations will occur during active gameplay sessions.⁸
2. **WebAssembly Canvas Limitations:** Compose Multiplatform for WASM is still in an active development phase, which may cause compatibility issues in specific mobile browsers.³¹
 - *Mitigation:* Thorough fallback testing and optimizing the rendering pipeline to a lower resolution on mobile web browsers.²² The web build will be strictly positioned as a marketing demo rather than the primary play environment.²²
3. **Inaccurate Touch Input:** At extremely high speeds, players might tap slightly off-center from an active node, leading to unfair time penalties and frustration.⁵
 - *Mitigation:* The physical touch box (hitbox) of the daemon nodes will be configured 20% larger than the visual sprite, providing higher player tolerance and a smoother flow experience.⁵

10.8 Definition of Done for the First Vertical Slice

Technical and Functional Acceptance Criteria

The first vertical slice of GRID_BREAKER is officially considered "done" when it meets the following objectively measurable criteria:

- **Performance and Framerate:** The game engine maintains a stable, uninterrupted frame rate of 60 frames per second on a physical iPhone 15 and a Samsung Galaxy S23 during an uninterrupted three-minute play session.⁷
- **Mechanical Integration:** The three basic daemon types (Standard, Armored, Firewall Bomb) spawn correctly according to the mathematical scaling matrix, react accurately to one or two taps respectively, and the bomb triggers an immediate game-over screen upon touch.⁵

- **Input Responsiveness:** The latency between a physical touch on the screen and the start of the visual hit-flash is under 16 milliseconds (within one frame).¹¹
- **Upgrade Loop:** The player can spend earned Credits in the terminal interface, permanently increasing maximum RAM capacity, with this new value correctly applied at the start of the next play session.¹⁸
- **Data Integrity:** High scores are successfully synchronized with the backend proxy upon session completion, validated via database verification.³

Works cited

1. 323 Whack Mole Game Stock Vectors and Vector Art - Shutterstock, accessed on June 7, 2026,
https://www.shutterstock.com/search/whack-mole-game?image_type=vector
2. TapTapBlitz - Apps on Google Play, accessed on June 7, 2026,
<https://play.google.com/store/apps/details?id=com.littlebigstudio.taptapblitz>
3. Naamloos document-4.pdf
4. The Hacking Minigame Is Too Complicated/Time Consuming :: Heart of the Machine Gameplay Discussion - Steam Community, accessed on June 7, 2026,
<https://steamcommunity.com/app/2001070/discussions/0/828204634466157129/?l=greek&ctp=2>
5. Whack-A-Mole Game Project Report | PDF | Graphics Processing Unit | Software - Scribd, accessed on June 7, 2026,
<https://www.scribd.com/document/816602736/Bharat-c-report-1>
6. An Interactive Physical-Cognitive Game-Based Training System Using Kinect for Older Adults: Development and Usability Study - PMC, accessed on June 7, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC8581754/>
7. How I Built a Roguelike RPG Card Game with Compose Multiplatform (and Skipped Traditional Engines) | by Clifton Robinson | Medium, accessed on June 7, 2026,
<https://medium.com/@cliffrob25/how-i-built-a-roguelike-rpg-with-compose-multiplatform-and-skipped-traditional-engines-c19c3fc6e310>
8. [Project] I built a performant Isometric Game Engine using 100% Pure Kotlin and Compose, accessed on June 7, 2026,
https://www.reddit.com/r/Kotlin/comments/1pss0s5/project_i_built_a_performant_isometric_game/
9. Whack-a-mole Online Learning: Physics-Informed Neural Network for Intraday Implied Volatility Surface - ResearchGate, accessed on June 7, 2026,
https://www.researchgate.net/publication/385560227_Whack-a-mole_Online_Learning_Physics-Informed_Neural_Network_for_Intraday_Implied_Volatility_Surface
10. Game feel - Wikipedia, accessed on June 7, 2026,
https://en.wikipedia.org/wiki/Game_feel
11. Game Feel: A Beginner's Guide - Game Design Skills, accessed on June 7, 2026,
<https://gamedesignskills.com/game-design/game-feel/>
12. Making Sense of 'Game Feel' through Affective Science - DiGRA Digital Library, accessed on June 7, 2026,

- <https://dl.digra.org/index.php/dl/article/download/2223/2220/2238>
13. New Retro Arcade Neon PC Game Review - Impulse Gamer, accessed on June 7, 2026, <https://www.impulsegamer.com/new-retro-arcade-neon-pc-game-review/>
 14. FL Tron - Classic Snake vs Tron Arcade Game - Seele AI, accessed on June 7, 2026, <https://www.seeles.ai/games/casual/fl-tron-classic-snake-vs-tron-arcade-game>
 15. CyberDrive 2077 - App Store, accessed on June 7, 2026, <https://apps.apple.com/us/app/cyberdrive-2077/id1401394993>
 16. Flip Rush - Impossible Game - App Store - Apple, accessed on June 7, 2026, <https://apps.apple.com/pl/app/flip-rush-impossible-game/id6758917514>
 17. Neon Commons - Apps on Google Play, accessed on June 7, 2026, <https://play.google.com/store/apps/details?id=com.neoncommons.social.space>
 18. Cyberpunk 2077 - How Hacking Works - YouTube, accessed on June 7, 2026, <https://www.youtube.com/watch?v=QepZc2Q1uxQ>
 19. [Cyberpunk 2077] Breach Protocol / Hacking Mini-Game Guide | Shortened Version of Previous Video - YouTube, accessed on June 7, 2026, <https://www.youtube.com/watch?v=1ffmb7tYNxo>
 20. Phantom Trigger: Neon Nightmares and Heaps of Hacking (and Slashing), accessed on June 7, 2026, <https://bigbossbattle.com/phantom-trigger-neon-nightmares-and-heaps-of-hacking-and-slashing/>
 21. Snake by PierreBDL, accessed on June 7, 2026, <https://pierrebdl.itch.io/snake>
 22. Neon Rocket Dash - Neon Arcade Rocket Game for Mobile and Desktop - vitaluniapp, accessed on June 7, 2026, <https://www.vitaluniapp.com/neon-rocket-dash.html>
 23. Orbit Hop: Neon Tap Arcade - Apps on Google Play, accessed on June 7, 2026, <https://play.google.com/store/apps/details?id=com.flavorgames.orbithop>
 24. Whack An Alien Mole Invader - Smash The Cute Miner Invaders From Mars! - App Store, accessed on June 7, 2026, <https://apps.apple.com/mx/app/whack-an-alien-mole-invader-smash-the-cute/id1021942140?l=en-GB>
 25. Whack a Bunny - Hole Puzzle 2D - App Store, accessed on June 7, 2026, <https://apps.apple.com/rs/app/whack-a-bunny-hole-puzzle-2d/id1532870066>
 26. Whack An Alien Mole Invader - Smash The Cute Miner Invaders From Mars! App - App Store, accessed on June 7, 2026, <https://apps.apple.com/us/app/whack-an-alien-mole-invader-smash-the-cute/id1021942140>
 27. Windows Excel Gevorderden 2013 | X-Cel, accessed on June 7, 2026, <https://www.x-cel.nl/module/win-exc-gev-les-1-2013/>
 28. Principles of Virtual Sensation - Game Developer, accessed on June 7, 2026, <https://www.gamedeveloper.com/design/principles-of-virtual-sensation>
 29. What is Skip?, accessed on June 7, 2026, <https://skip.dev/docs/>
 30. Compose Multiplatform - Beautiful UIs Everywhere - Kotlin, accessed on June 7, 2026, <https://kotlinlang.org/compose-multiplatform/>
 31. Compose Multiplatform for web goes Beta in the 1.9.0 release : r/Kotlin - Reddit,

- accessed on June 7, 2026,
https://www.reddit.com/r/Kotlin/comments/1nnpo0q/compose_multiplatform_for_web_goes_beta_in_the/
32. Kotlin Ecosystem AMA – December 11 (3–7 pm CET) - Reddit, accessed on June 7, 2026,
https://www.reddit.com/r/Kotlin/comments/1pjs3bd/kotlin_ecosystem_ama_dece_mber_11_37_pm_cet/
 33. Embedded games in mobile apps: Make your app more attractive | GoodRequest, accessed on June 7, 2026,
<https://www.goodrequest.com/blog/embedded-games-in-mobile-apps-make-your-app-more-attractive>
 34. Building Once, Deploying Everywhere: How Compose Multiplatform Changes the Game | by Malaikaakhtar | Medium, accessed on June 7, 2026,
<https://medium.com/@malaikaakhtar808/building-once-deploying-everywhere-how-compose-multiplatform-changes-the-game-081082f158ee>
 35. Skip - GitHub, accessed on June 7, 2026, <https://github.com/skiptools>
 36. Skip: One Swift Codebase. Two Native Platforms., accessed on June 7, 2026,
<https://skip.dev/>
 37. Learn How Skip Simplifies Cross-Platform App Development By Transpiling Swift To Kotlin, accessed on June 7, 2026,
<https://www.superbcrew.com/learn-how-skip-simplifies-cross-platform-app-development-by-transpiling-swift-to-kotlin/>
 38. Skip 1.0 released: build iOS and Android apps from a single Swift codebase - Reddit, accessed on June 7, 2026,
https://www.reddit.com/r/swift/comments/1etvo94/skip_10_released_build_ios_and_android_apps_from/
 39. Taptic Reflex – Speed Game - Apps on Google Play, accessed on June 7, 2026,
<https://play.google.com/store/apps/details?id=com.taptic.reflex>
 40. Fable: New Game (2026) — What It Is, Why It's Hype, and How to Make Your Own Fable-Style Game Free - Rosebud AI, accessed on June 7, 2026,
<https://lab.rosebud.ai/blog/fable-2026-everything-we-know-about-the-new-game---and-how-to-play-a-fable-style-rpg-for-free>
 41. What are your Core Pillars? and How do you Stick to them? : r/gamedesign - Reddit, accessed on June 7, 2026,
https://www.reddit.com/r/gamedesign/comments/apxo1a/what_are_your_core_pillars_and_how_do_you_stick/
 42. I built a deck-building roguelike card game entirely in Compose Multiplatform — targeting Android, iOS, Desktop, and Meta Quest VR : r/Kotlin - Reddit, accessed on June 7, 2026,
https://www.reddit.com/r/Kotlin/comments/1rwqfuc/i_built_a_deckbuilding_rogue_ike_card_game/
 43. GitHub - pandulapeter/kubriko: A game engine based on Compose Multiplatform for 2D Android, iOS, Desktop, and Web projects., accessed on June 7, 2026,
<https://github.com/pandulapeter/kubriko>
 44. visrm/Whack-A-Mole-Game - GitHub, accessed on June 7, 2026,

<https://github.com/vism/Whack-A-Mole-Game>